

Linear Equations

- Problem Formulation
- Conditioning of Linear Equation Systems
- Gauss Elimination
- Linear Equations with Band-Structured Matrix
- Linear Equations with Positive Definite Matrices
- QR-Decomposition

Contents

- Problem Formulation
- Conditioning of Linear Equation Systems
- Gauss Elimination
- Linear Equations with Band-Structured Matrix
- Linear Equations with Positive Definite Matrices
- QR-Decomposition

Problem Formulation

Given a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$. We are searching for solutions of the linear equation system

$$Ax = b .$$

Names:

- For $m < n$: “under-determined linear equation system”
- For $m = n$: “quadratic linear equation system”
- For $m > n$: “over-determined linear equation system”

Problem Formulation

Given a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$. We are searching for solutions of the linear equation system

$$Ax = b .$$

Names:

- For $m < n$: “under-determined linear equation system”
- For $m = n$: “quadratic linear equation system”
- For $m > n$: “over-determined linear equation system”

Problem Formulation

Given a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$. We are searching for solutions of the linear equation system

$$Ax = b .$$

Names:

- For $m < n$: “under-determined linear equation system”
- For $m = n$: “quadratic linear equation system”
- For $m > n$: “over-determined linear equation system”

Problem Formulation

Given a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$. We are searching for solutions of the linear equation system

$$Ax = b .$$

Names:

- For $m < n$: “under-determined linear equation system”
- For $m = n$: “quadratic linear equation system”
- For $m > n$: “over-determined linear equation system”

Known Results from Linear Algebra

The equation system $Ax = b$ has a solution if and only if $\text{rank}(A) = \text{rank}(A, b)$.

This condition can, e.g., be checked with Gram-Schmidt algorithms.

For the quadratic case $n = m$, the following statements are equivalent:

- $Ax = b$ has the unique solution x .
- $\text{rank}(A) = n$.
- $\det(A) \neq 0$.
- All eigenvalues of A are different from 0.

Known Results from Linear Algebra

The equation system $Ax = b$ has a solution if and only if $\text{rank}(A) = \text{rank}(A, b)$.

This condition can, e.g., be checked with Gram-Schmidt algorithms.

For the quadratic case $n = m$, the following statements are equivalent:

- $Ax = b$ has the unique solution x .
- $\text{rank}(A) = n$.
- $\det(A) \neq 0$.
- All eigenvalues of A are different from 0.

Contents

- Problem Formulation
- **Conditioning of Linear Equation Systems**
- Gauss Elimination
- Linear Equations with Band-Structured Matrix
- Linear Equations with Positive Definite Matrices
- QR-Decomposition

Overview: Perturbation Theory

Whenever we solve equations of the form $Ax = b$ numerically, we have to take into account two possible sources of errors:

- Errors that are due to working with inexact numerical values for A and b .
- Errors that are due to numerical rounding problems when implementing the algorithm based on finite precision arithmetics.

Matrix Norms

Recall that any vector norm $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ can be generalized for matrices by defining

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|},$$

for any $A \in \mathbb{R}^{m \times n}$.

Simple examples:

- $\|A\|_1 = \max_j \sum_{i=0}^m |A_{i,j}|.$
- $\|A\|_\infty = \max_i \sum_{j=0}^n |A_{i,j}|.$

The Spectral Norm

The eigenvalues $\lambda_1, \dots, \lambda_n \in \mathbb{C}$ of a square matrix $A \in \mathbb{R}^{n \times n}$ are the roots of the characteristic polynomial $p(\lambda) = \det(A - \lambda I)$.

The matrix 2-norm is given by

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \max \left\{ \sqrt{\lambda} \mid \det(A^T A - \lambda I) = 0 \right\} ,$$

If A is symmetric this expression for the 2-norm can be simplified

$$\|A\|_2 = \max \{ |\lambda| \mid \det(A - \lambda I) = 0 \} .$$

The Spectral Norm

The eigenvalues $\lambda_1, \dots, \lambda_n \in \mathbb{C}$ of a square matrix $A \in \mathbb{R}^{n \times n}$ are the roots of the characteristic polynomial $p(\lambda) = \det(A - \lambda I)$.

The matrix 2-norm is given by

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \max \left\{ \sqrt{\lambda} \mid \det(A^T A - \lambda I) = 0 \right\} ,$$

If A is symmetric this expression for the 2-norm can be simplified

$$\|A\|_2 = \max \{ |\lambda| \mid \det(A - \lambda I) = 0 \} .$$

A Useful Matrix-Norm Inequality

If $\|\cdot\|$ is a matrix norm (assume $\|I\| = 1$) and $A \in \mathbb{R}^{n \times n}$ a given matrix whose norm satisfies $\|A\| < 1$, then the matrix $I + A$ is invertible and we have

$$\|(I + A)^{-1}\| \leq \frac{1}{1 - \|A\|}$$

Proof: From the inequality

$$\|(I + A)x\| \geq \underbrace{(1 - \|A\|)}_{>0} \|x\|$$

we conclude that we have $(I + A)x \neq 0$ for all vectors $x \neq 0$; that is, $I + A$ is invertible. Moreover, we have

$$1 = \|(I + A)(I + A)^{-1}\| \geq \|(I + A)^{-1}\|(1 - \|A\|),$$

which implies the statement.

A Useful Matrix-Norm Inequality

If $\|\cdot\|$ is a matrix norm (assume $\|I\| = 1$) and $A \in \mathbb{R}^{n \times n}$ a given matrix whose norm satisfies $\|A\| < 1$, then the matrix $I + A$ is invertible and we have

$$\|(I + A)^{-1}\| \leq \frac{1}{1 - \|A\|}$$

Proof: From the inequality

$$\|(I + A)x\| \geq \underbrace{(1 - \|A\|)}_{>0} \|x\|$$

we conclude that we have $(I + A)x \neq 0$ for all vectors $x \neq 0$; that is, $I + A$ is invertible. Moreover, we have

$$1 = \|(I + A)(I + A)^{-1}\| \geq \|(I + A)^{-1}\|(1 - \|A\|),$$

which implies the statement.

Condition Numbers

Theorem:

Let x denote the solution of the invertible and quadratic linear equation system $Ax = b$ and let δx be such that

$$(A + \delta A)(x + \delta x) = b + \delta b .$$

assuming $\|\delta A\| < \frac{1}{\|A^{-1}\|}$ and $\|b\| \neq 0$ as well as $\|A\| \neq 0$. Then we have

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\text{cond}(A)}{1 - \text{cond}(A) \frac{\|\delta A\|}{\|A\|}} \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right) \quad \text{with} \quad \text{cond}(A) = \|A\| \cdot \|A^{-1}\| .$$

Interpretation:

The condition number $\text{cond}(A)$ can be interpreted as an error amplification factor,

$$\frac{\|\delta x\|}{\|x\|} \approx \text{cond}(A) \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right) .$$

Condition Numbers

Theorem:

Let x denote the solution of the invertible and quadratic linear equation system $Ax = b$ and let δx be such that

$$(A + \delta A)(x + \delta x) = b + \delta b .$$

assuming $\|\delta A\| < \frac{1}{\|A^{-1}\|}$ and $\|b\| \neq 0$ as well as $\|A\| \neq 0$. Then we have

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\text{cond}(A)}{1 - \text{cond}(A) \frac{\|\delta A\|}{\|A\|}} \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right) \quad \text{with} \quad \text{cond}(A) = \|A\| \cdot \|A^{-1}\| .$$

Interpretation:

The condition number $\text{cond}(A)$ can be interpreted as an error amplification factor,

$$\frac{\|\delta x\|}{\|x\|} \approx \text{cond}(A) \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right) .$$

Proof

From $Ax = b$ and the equation

$$(A + \delta A)(x + \delta x) = b + \delta b .$$

we find $\delta x = (A + \delta A)^{-1}(\delta b - \delta Ax)$ (recall that $\|\delta A\| \leq \frac{1}{\|A^{-1}\|}$ implies that $A + \delta A$ is invertible). This yields the estimate

$$\begin{aligned}\|\delta x\| &\leq \|(A + \delta A)^{-1}\| (\|\delta b\| + \|\delta A\| \|x\|) \\ &\leq \|(I + A^{-1}\delta A)^{-1}\| \|A^{-1}\| (\|\delta b\| + \|\delta A\| \|x\|) \\ &\leq \frac{\|A^{-1}\|}{1 - \|A^{-1}\| \|\delta A\|} (\|\delta b\| + \|\delta A\| \|x\|)\end{aligned}$$

(Last step has used the matrix-norm ineq. from previous slides)

Proof (continued)

Because we have $Ax = b$, it follows that $\|A\|\|x\| \geq \|b\|$ and

$$\begin{aligned}\|\delta x\| &\leq \frac{\|A^{-1}\|}{1 - \|A^{-1}\|\|\delta A\|} (\|\delta b\| + \|\delta A\|\|x\|) \\ &\leq \frac{\|A^{-1}\|\|A\|\|x\|}{1 - \|A^{-1}\|\|\delta A\|} \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right).\end{aligned}$$

In the last step we substitute $\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$:

$$\|\delta x\| \leq \frac{\text{cond}(A)\|x\|}{1 - \text{cond}(A)\frac{\|\delta A\|}{\|A\|}} \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right).$$

The statement of the theorem follows after dividing by $\|x\|$.

Contents

- Problem Formulation
- Conditioning of Linear Equation Systems
- **Gauss Elimination**
- Linear Equations with Band-Structured Matrix
- Linear Equations with Positive Definite Matrices
- QR-Decomposition

Triangular Matrices

If the matrix A is triangular, i.e.,

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ 0 & a_{2,2} & \dots & a_{2,n} \\ \vdots & \ddots & \ddots & \\ 0 & \dots & 0 & a_{n,n} \end{pmatrix},$$

the system Ax can be solved by a “backwards substitution”:

$$x_n = \frac{b_n}{a_{n,n}} \quad \text{and} \quad x_j = \frac{1}{a_{j,j}} \left(b_j - \sum_{k=j+1}^n a_{j,k} x_k \right)$$

for $j = n - 1, \dots, 1$. (Complexity: $\mathbf{O}(n^2)$.)

Gauss Elimination

For general matrices A the main idea is to transform the matrix step by step into an upper triangular matrix. For this aim, three types of operations can be applied:

- We may swap (permute) rows of A if necessary.
- We may swap (permute) columns as long as we re-enumerate the unknowns x_j , too.
- We may multiply one row by a scalar factor and add it to another row.

Gauss Elimination

For general matrices A the main idea is to transform the matrix step by step into an upper triangular matrix. For this aim, three types of operations can be applied:

- We may swap (permute) rows of A if necessary.
- We may swap (permute) columns as long as we re-enumerate the unknowns x_j , too.
- We may multiply one row by a scalar factor and add it to another row.

Gauss Elimination

For general matrices A the main idea is to transform the matrix step by step into an upper triangular matrix. For this aim, three types of operations can be applied:

- We may swap (permute) rows of A if necessary.
- We may swap (permute) columns as long as we re-enumerate the unknowns x_j , too.
- We may multiply one row by a scalar factor and add it to another row.

Gauss Elimination

For general matrices A the main idea is to transform the matrix step by step into an upper triangular matrix. For this aim, three types of operations can be applied:

- We may swap (permute) rows of A if necessary.
- We may swap (permute) columns as long as we re-enumerate the unknowns x_j , too.
- We may multiply one row by a scalar factor and add it to another row.

Gauss Elimination

In the first step of the Gauss elimination procedure is as follows:

- Permute the rows of A such that $a_{1,1} \neq 0$.
- For all row indices $j = 2, \dots, n$: subtract $q_j = \frac{a_{j,1}}{a_{1,1}}$ times the first row from the j -th row.
- The result of these operations is a matrix of the form

$$C = \begin{pmatrix} c_{1,1} & c_{1,2} & \dots & c_{1,n} \\ 0 & c_{2,2} & \dots & c_{2,n} \\ \vdots & \vdots & & \vdots \\ 0 & c_{n,2} & \dots & c_{n,n} \end{pmatrix}.$$

Gauss Elimination

In the first step of the Gauss elimination procedure is as follows:

- Permute the rows of A such that $a_{1,1} \neq 0$.
- For all row indices $j = 2, \dots, n$: subtract $q_j = \frac{a_{j,1}}{a_{1,1}}$ times the first row from the j -th row.
- The result of these operations is a matrix of the form

$$C = \begin{pmatrix} c_{1,1} & c_{1,2} & \dots & c_{1,n} \\ 0 & c_{2,2} & \dots & c_{2,n} \\ \vdots & \vdots & & \vdots \\ 0 & c_{n,2} & \dots & c_{n,n} \end{pmatrix}.$$

Gauss Elimination

In the first step of the Gauss elimination procedure is as follows:

- Permute the rows of A such that $a_{1,1} \neq 0$.
- For all row indices $j = 2, \dots, n$: subtract $q_j = \frac{a_{j,1}}{a_{1,1}}$ times the first row from the j -th row.
- The result of these operations is a matrix of the form

$$C = \begin{pmatrix} c_{1,1} & c_{1,2} & \dots & c_{1,n} \\ 0 & c_{2,2} & \dots & c_{2,n} \\ \vdots & \vdots & & \vdots \\ 0 & c_{n,2} & \dots & c_{n,n} \end{pmatrix}.$$

Gauss Elimination

The first step of the Gauss elimination procedure can also be summarized in the form:

$$C = G_1 P_1 A \quad \text{and} \quad d = G_1 P_1 b$$

such that the equation system $Ax = b$ is equivalent to the equation system $Cx = d$. Here, P_1 is a permutation matrix and G_1 a Frobenius matrix of the form

$$G_1 = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ q_2 & 1 & 0 & \dots & 0 \\ q_3 & 0 & 1 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ q_n & 0 & \dots & 0 & 1 \end{pmatrix}.$$

Gauss Elimination

In the second step of the Gauss elimination we apply the same transformation strategy to the remaining $(n - 1) \times (n - 1)$ dense block such that

$$E = G_2 P_2 G_1 P_1 A \quad \text{and} \quad f = G_2 P_2 G_1 P_1 b$$

such that the equation system $Ax = b$ is equivalent to the equation system $Ex = f$. Here, E has the form

$$E = \begin{pmatrix} c_{1,1} & c_{1,2} & c_{1,3} & \dots & c_{1,n} \\ 0 & e_{2,2} & e_{2,3} & \dots & e_{2,n} \\ 0 & 0 & e_{3,3} & \dots & e_{3,n} \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & e_{n,3} & \dots & e_{n,n} \end{pmatrix}.$$

Gauss Elimination

If we keep on applying the same strategy we end up with a triangular matrix

$$R = G_{n-1}P_{n-1} \dots G_1P_1A .$$

and a vector $s = G_{n-1}P_{n-1} \dots G_1P_1b$ such that the equation system $Ax = b$ is equivalent to the equation system $Rx = s$. The triangular system can then be solved by backwards substitution. The complexity of computing R is of order $\mathcal{O}(n^3)$.

Gauss Elimination

If we keep on applying the same strategy we end up with a triangular matrix

$$R = G_{n-1}P_{n-1} \dots G_1P_1A .$$

and a vector $s = G_{n-1}P_{n-1} \dots G_1P_1b$ such that the equation system $Ax = b$ is equivalent to the equation system $Rx = s$. The triangular system can then be solved by backwards substitution. The complexity of computing R is of order $\mathbf{O}(n^3)$.

LR Decomposition

We assume for simplicity that we do not have to permute the rows of A .

The matrix

$$L = G_1^{-1}G_2^{-1} \dots G_{n-1}^{-1} = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ l_{2,1} & 1 & 0 & \dots & 0 \\ l_{3,1} & l_{3,2} & 1 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ l_{n,1} & l_{n,2} & \dots & l_{n,n-1} & 1 \end{pmatrix}$$

is lower triangular and satisfies $LR = A$ by construction.

Uniqueness of LR-Decomposition

The LR-Decomposition is unique: if there were two LR decompositions

$$A = L_1 R_1 = L_2 R_2$$

we would have

$$L_2^{-1} L_1 = R_2 R_1^{-1} = I ,$$

since the matrix $L_2^{-1} L_1$ is lower triangular (with ones on the diagonal) and R is an upper triangular matrix. Thus, we have $R_1 = R_2$ and $L_1 = L_2$.

Application of LR Decomposition in Practice

In practice, we are often in the situation that we want to solve multiple linear equation systems of the form

$$Ax_i = b_i, \quad i = 1, 2, \dots, N$$

for changing vectors $b_i \in \mathbb{R}^n$. A naive implementation of Gauss elimination would lead to the complexity $\mathbf{O}(N \cdot n^3)$ for computing $x_1, \dots, x_N$.

A major speed-up can be obtained by storing the LR-decomposition, $A = LR$, of the matrix A and then compute

$$x_1 = R^{-1}L^{-1}b_1, \quad x_2 = R^{-1}L^{-2}b_2 \dots$$

Now, the complexity is $\mathbf{O}(n^3 + N \cdot n^2)$.

Application of LR Decomposition in Practice

In practice, we are often in the situation that we want to solve multiple linear equation systems of the form

$$Ax_i = b_i, \quad i = 1, 2, \dots, N$$

for changing vectors $b_i \in \mathbb{R}^n$. A naive implementation of Gauss elimination would lead to the complexity $\mathbf{O}(N \cdot n^3)$ for computing $x_1, \dots, x_N$.

A major speed-up can be obtained by storing the LR-decomposition, $A = LR$, of the matrix A and then compute

$$x_1 = R^{-1}L^{-1}b_1, \quad x_2 = R^{-1}L^{-2}b_2 \dots$$

Now, the complexity is $\mathbf{O}(n^3 + N \cdot n^2)$.

Computation of Determinants

If a LR-decomposition of the matrix $A \in \mathbb{R}^{n \times n}$ is known, the determinant can be computed from

$$\det(A) = \det(LR) = \det(L)\det(R) = \det(R) = \prod_{i=1}^n R_{ii} .$$

Contents

- Problem Formulation
- Conditioning of Linear Equation Systems
- Gauss Elimination
- **Linear Equations with Band-Structured Matrix**
- Linear Equations with Positive Definite Matrices
- QR-Decomposition

Band-Structured Matrices

A band-structured matrix is a matrix whose components satisfy

$$a_{j,k} = 0 \quad \text{for} \quad k < j - m_l \quad \text{or} \quad k > j + m_r .$$

Here m_l and m_r are called the widths of the band-matrix A .

Examples:

- For $m_l = 0$ and $m_r \leq n - 1$: upper triangular matrix.
- For $m_r = 0$ and $m_l \leq n - 1$: lower triangular matrix.
- For $m_l = 1$ and $m_r = 1$: tridiagonal matrix.

Band-Structured Matrices

A band-structured matrix is a matrix whose components satisfy

$$a_{j,k} = 0 \quad \text{for} \quad k < j - m_l \quad \text{or} \quad k > j + m_r .$$

Here m_l and m_r are called the widths of the band-matrix A .

Examples:

- For $m_l = 0$ and $m_r \leq n - 1$: upper triangular matrix.
- For $m_r = 0$ and $m_l \leq n - 1$: lower triangular matrix.
- For $m_l = 1$ and $m_r = 1$: tridiagonal matrix.

Gauss Elimination for Band-Structured Matrices

If we do not have to permute rows, equation systems of the form

$$Ax = b$$

can be solved with Gauss-elimination, too, while exploiting the band-structure. The complexity is in this case

$$\mathbf{O}(nm_l m_r) + \mathbf{O}(n(m_l + m_r)) .$$

Example: Tridiagonal Matrices

For the case that A is a tridiagonal matrix of the form

$$A = \begin{pmatrix} a_1 & b_1 & & \\ c_2 & \ddots & \ddots & \\ & \ddots & \ddots & b_{n-1} \\ & & c_n & a_n \end{pmatrix}$$

the matrices L and R have the form

$$L = \begin{pmatrix} 1 & & & \\ \gamma_2 & \ddots & & \\ & \ddots & 1 & \\ & & \gamma_n & 1 \end{pmatrix} \quad \text{and} \quad R = \begin{pmatrix} \alpha_1 & b_1 & & \\ & \ddots & \ddots & \\ & & \alpha_{n-1} & b_{n-1} \\ & & & \alpha_n \end{pmatrix}.$$

Example: Tridiagonal Matrices

The coefficients α_i and γ_i can be computed by the recursion

$$\begin{aligned}\alpha_1 &= a_1, \\ \gamma_i &= \frac{c_i}{\alpha_{i-1}}, \quad \text{and} \quad \alpha_i = a_i - \gamma_i b_i\end{aligned}$$

for $i = 2, \dots, n$.

Contents

- Problem Formulation
- Conditioning of Linear Equation Systems
- Gauss Elimination
- Linear Equations with Band-Structured Matrix
- **Linear Equations with Positive Definite Matrices**
- QR-Decomposition

Symmetric Positive Definite Matrices

A matrix $A \in \mathbb{R}^{n \times n}$ is called symmetric positive definite if we have $A = A^T$ and

$$\forall v \in \mathbb{R}^n \setminus \{0\}, \quad v^T A v > 0 .$$

Equation systems of the form $Ax = b$ with A being symmetric positive semi-definite are solved by representing A in the form

$$A = LDL^T ,$$

where $L \in \mathbb{R}^{n \times n}$ is a lower diagonal matrix with ones on the diagonal and $D \in \mathbb{R}^{n \times n}$ a diagonal matrix.

Symmetric Positive Definite Matrices

A matrix $A \in \mathbb{R}^{n \times n}$ is called symmetric positive definite if we have $A = A^T$ and

$$\forall v \in \mathbb{R}^n \setminus \{0\}, \quad v^T A v > 0 .$$

Equation systems of the form $Ax = b$ with A being symmetric positive semi-definite are solved by representing A in the form

$$A = LDL^T ,$$

where $L \in \mathbb{R}^{n \times n}$ is a lower diagonal matrix with ones on the diagonal and $D \in \mathbb{R}^{n \times n}$ a diagonal matrix.

Cholesky Algorithm (LDL^T -Variant)

Let us work out the matrix product

$$\begin{aligned}
 LDL^T &= \begin{pmatrix} 1 & & & \\ L_{2,1} & 1 & & \\ L_{3,1} & L_{3,2} & 1 & \\ \vdots & \vdots & & \ddots \end{pmatrix} \begin{pmatrix} D_1 & & & \\ & D_2 & & \\ & & D_3 & \\ & & & \ddots \end{pmatrix} \begin{pmatrix} 1 & & & \\ L_{2,1} & 1 & & \\ L_{3,1} & L_{3,2} & 1 & \\ \vdots & \vdots & & \ddots \end{pmatrix}^T \\
 &= \begin{pmatrix} D_1 & & & \\ L_{2,1}D_1 & L_{2,1}^2D_1 + D_2 & & \\ L_{3,1}D_1 & L_{3,1}L_{2,1}D_1 + L_{3,2}D_2 & L_{3,1}^2D_1 + L_{3,2}^2D_2 + D_3 & \\ \vdots & \vdots & & \ddots \end{pmatrix} \begin{matrix} \\ \\ \text{symmetric} \\ \end{matrix}
 \end{aligned}$$

Cholesky Algorithm (LDL^T -Variant)

We want to determine L and D such that

$$A = \begin{pmatrix} D_1 & & & & \\ L_{2,1}D_1 & L_{2,1}^2D_1 + D_2 & & & \\ L_{3,1}D_1 & L_{3,1}L_{2,1}D_1 + L_{3,2}D_2 & L_{3,1}^2D_1 + L_{3,2}^2D_2 + D_3 & & \\ \vdots & \vdots & & \ddots & \\ & & & & \ddots \end{pmatrix} \quad \text{symmetric}$$

- We first set $D_1 = A_{1,1} > 0$ (since A is positive definite).
- We can find $L_{2,1} = A_{2,1}/D_1$.
- The diagonal entry $D_2 = A_{2,2} - L_{2,1}^2D_1 > 0$ must be positive.
- General recursion (for $i > j$):

$$D_{j,j} = A_{j,j} - \sum_{k=1}^{j-1} L_{j,k}^2 D_k \quad \text{and} \quad L_{i,j} = \frac{1}{D_j} \left(A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} D_k \right).$$

Cholesky Algorithm (LDL^T -Variant)

We want to determine L and D such that

$$A = \begin{pmatrix} D_1 & & & & \\ L_{2,1}D_1 & L_{2,1}^2D_1 + D_2 & & & \\ L_{3,1}D_1 & L_{3,1}L_{2,1}D_1 + L_{3,2}D_2 & L_{3,1}^2D_1 + L_{3,2}^2D_2 + D_3 & & \\ \vdots & \vdots & & \ddots & \\ & & & & \ddots \end{pmatrix} \quad \text{symmetric}$$

- We first set $D_1 = A_{1,1} > 0$ (since A is positive definite).
- We can find $L_{2,1} = A_{2,1}/D_1$.
- The diagonal entry $D_2 = A_{2,2} - L_{2,1}^2D_1 > 0$ must be positive.
- General recursion (for $i > j$):

$$D_{j,j} = A_{j,j} - \sum_{k=1}^{j-1} L_{j,k}^2 D_k \quad \text{and} \quad L_{i,j} = \frac{1}{D_j} \left(A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} D_k \right).$$

Cholesky Algorithm (LDL^T -Variant)

We want to determine L and D such that

$$A = \begin{pmatrix} D_1 & & & & \\ L_{2,1}D_1 & L_{2,1}^2D_1 + D_2 & & & \\ L_{3,1}D_1 & L_{3,1}L_{2,1}D_1 + L_{3,2}D_2 & L_{3,1}^2D_1 + L_{3,2}^2D_2 + D_3 & & \\ \vdots & \vdots & & \ddots & \\ & & & & \ddots \end{pmatrix} \quad \text{symmetric}$$

- We first set $D_1 = A_{1,1} > 0$ (since A is positive definite).
- We can find $L_{2,1} = A_{2,1}/D_1$.
- The diagonal entry $D_2 = A_{2,2} - L_{2,1}^2D_1 > 0$ must be positive.
- General recursion (for $i > j$):

$$D_{j,j} = A_{j,j} - \sum_{k=1}^{j-1} L_{j,k}^2 D_k \quad \text{and} \quad L_{i,j} = \frac{1}{D_j} \left(A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} D_k \right).$$

Cholesky Algorithm (LDL^T -Variant)

We want to determine L and D such that

$$A = \begin{pmatrix} D_1 & & & \\ L_{2,1}D_1 & L_{2,1}^2D_1 + D_2 & & \\ L_{3,1}D_1 & L_{3,1}L_{2,1}D_1 + L_{3,2}D_2 & L_{3,1}^2D_1 + L_{3,2}^2D_2 + D_3 & \\ \vdots & \vdots & & \ddots \end{pmatrix} \quad \text{symmetric}$$

- We first set $D_1 = A_{1,1} > 0$ (since A is positive definite).
- We can find $L_{2,1} = A_{2,1}/D_1$.
- The diagonal entry $D_2 = A_{2,2} - L_{2,1}^2D_1 > 0$ must be positive.
- General recursion (for $i > j$):

$$D_{j,j} = A_{j,j} - \sum_{k=1}^{j-1} L_{j,k}^2 D_k \quad \text{and} \quad L_{i,j} = \frac{1}{D_j} \left(A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} D_k \right).$$

Cholesky Algorithm (LDL^T -Variant)

We want to determine L and D such that

$$A = \begin{pmatrix} D_1 & & & & \\ L_{2,1}D_1 & L_{2,1}^2D_1 + D_2 & & & \\ L_{3,1}D_1 & L_{3,1}L_{2,1}D_1 + L_{3,2}D_2 & L_{3,1}^2D_1 + L_{3,2}^2D_2 + D_3 & & \\ \vdots & \vdots & & \ddots & \\ & & & & \ddots \end{pmatrix} \quad \text{symmetric}$$

- We first set $D_1 = A_{1,1} > 0$ (since A is positive definite).
- We can find $L_{2,1} = A_{2,1}/D_1$.
- The diagonal entry $D_2 = A_{2,2} - L_{2,1}^2D_1 > 0$ must be positive.
- General recursion (for $i > j$):

$$D_{j,j} = A_{j,j} - \sum_{k=1}^{j-1} L_{j,k}^2 D_k \quad \text{and} \quad L_{i,j} = \frac{1}{D_j} \left(A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} D_k \right).$$

Cholesky Decomposition

- Cholesky's algorithm works reliably for positive definite matrices, $D_{ii} > 0$. We could also set

$$A = \tilde{L}^T \tilde{L} \quad \text{with} \quad \tilde{L} = D^{\frac{1}{2}} L .$$

This is the “original” Cholesky decomposition ($\tilde{L}$ can be interpreted as a “square-root” of A)

- Variants of the above $L^T D L$ algorithm are applicable for symmetric indefinite matrices, but require permutation of rows/columns
- For the symmetric non-positive-definite matrix

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

the $L^T D L$ algorithm fails (but we could permute rows).

Contents

- Problem Formulation
- Conditioning of Linear Equation Systems
- Gauss Elimination
- Linear Equations with Band-Structured Matrix
- Linear Equations with Positive Definite Matrices
- **QR-Decomposition**

Orthonormal Basis

- Let $A = (a_1, a_2, a_3, \dots, a_m) \in \mathbb{R}^{n \times m}$ be a given matrix
- The vectors $a_1, a_2, \dots, a_m \in \mathbb{R}^n$ are the columns of A
- The vectors $q_1, q_2, \dots, q_r \in \mathbb{R}^n$ with $r \leq n$ are called an orthonormal basis if

$$\text{span}(q_1, q_2, \dots, q_r) = \text{span}(a_1, a_2, \dots, a_m) = \text{Im}(A)$$

and

$$\forall i, j \in \{1, 2, \dots, r\}, \quad q_i^T q_j = \delta_{i,j} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

- Notice that we must have $r = \dim(\text{Im}(A))$

QR Decomposition

- Regard the orthormal basis vectors as columns of a matrix Q ,

$$Q = (q_1, q_2, \dots, q_r)$$

- The matrix Q satisfies $Q^T Q = I$
- And we have $\text{Im}(A) = \text{Im}(Q)$
- On the next slides we will prove that there exist an upper triangular matrix R such that

$$A = QR .$$

This is called a QR decomposition of A .

Gram-Schmidt Algorithm

Input: m vectors $a_1, \dots, a_m \in \mathbb{R}^n$.

Gram-Schmidt Algorithm:

For $i = 1, \dots, m$:

- Orthogonalization. $\bar{q}_i = a_i - (q_1^T a_i)q_1 - \dots - (q_{i-1}^T a_i)q_{i-1}$.
- Test for linear dependence. If $\bar{q}_i = 0$, set $q_i = 0$ and skip next step.
- Normalization. $q_i = \frac{\bar{q}_i}{\|\bar{q}_i\|_H}$.

Optional: delete all 0s from the sequence $q_1, q_2, \dots, q_m$. This saves memory but we should keep an index list (or use a sparse matrix format).

Gram-Schmidt Algorithm

Input: m vectors $a_1, \dots, a_m \in \mathbb{R}^n$.

Gram-Schmidt Algorithm:

For $i = 1, \dots, m$:

- Orthogonalization. $\bar{q}_i = a_i - (q_1^T a_i)q_1 - \dots - (q_{i-1}^T a_i)q_{i-1}$.
- Test for linear dependence. If $\bar{q}_i = 0$, set $q_i = 0$ and skip next step.
- Normalization. $q_i = \frac{\bar{q}_i}{\|\bar{q}_i\|_H}$.

Optional: delete all 0s from the sequence $q_1, q_2, \dots, q_m$. This saves memory but we should keep an index list (or use a sparse matrix format).

Properties of Gram-Schmidt Algorithm

Gram-Schmidt constructs the $q_1, q_2, \dots, q_m$ (with 0s) such that

$$\begin{aligned}a_1 &= (a_1^\top q_1)q_1 \\a_2 &= (a_2^\top q_1)q_1 + (a_2^\top q_2)q_2 \\&\vdots \\a_m &= (a_m^\top q_1)q_1 + (a_m^\top q_2)q_2 + \dots + (a_m^\top q_m)q_m\end{aligned}\tag{1}$$

Moreover, we have $q_i^\top q_j = \delta_{i,j}$ (Exercise: proof by induction!)

QR Decomposition

$$\text{Write } a_1 = r_{1,1}q_1$$

$$a_2 = r_{1,2}q_1 + r_{2,2}q_2$$

$$\vdots$$

$$a_m = r_{1,m}q_1 + r_{2,m}q_2 + \dots + r_{m,m}q_m$$

with $r_{i,j} = a_i^\top q_j$, which yields $A = QR$ with

$$\underbrace{(a_1, a_2, \dots, a_m)}_{=A} = \underbrace{(q_1, q_2, \dots, q_m)}_{=Q} \underbrace{\begin{pmatrix} r_{1,1} & r_{1,2} & \dots & r_{1,m} \\ & r_{2,2} & \dots & r_{2,m} \\ & & \ddots & \vdots \\ & & & r_{m,m} \end{pmatrix}}_{=R}$$

Complete QR decomposition

Problem:

- we might have $\dim(\text{Im}(A)) = r < n$.
- Thus, if $r < n$, we have $Q^T Q = I$ but $Q Q^T \neq I$.

Remedy:

- apply Gram-Schmidt algorithm to the augmented matrix $\tilde{A} = [A, I]$
- delete all “zero-columns” from Q (and corresponding rows of R)
- this yields Q and R such that $QR = \tilde{A}$ and

$$Q \in \mathbb{R}^{n \times n}, \quad R = \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix} \quad \text{with} \quad \begin{cases} R_{11} \in \mathbb{R}^{r \times m} \\ R_{22} \in \mathbb{R}^{n-r \times n}, \end{cases}$$

where Q is orthogonal, R_{11} and R_{22} upper triangular (with full rank).

QR Decomposition

- In summary, Gram-Schmidt algorithm yields the representation

$$A = \begin{bmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \end{bmatrix} \begin{bmatrix} R_{11} \\ 0 \end{bmatrix}$$

- The equation system $Ax = b$ admits a solution if and only if

$$\begin{bmatrix} Q_{12}^T & Q_{22}^T \end{bmatrix} b = 0$$

- If the above condition holds, all solutions of $Ax = b$ satisfy

$$R_{11}x = r_1 \quad \text{with} \quad r_1 = \begin{bmatrix} Q_{11}^T & Q_{21}^T \end{bmatrix} b$$

- Solve by backward substitution! (always works, as R_{11} has full rank!)

Householder's QR Algorithm

- Gram-Schmidt algorithm are sometimes affected by cancellation / numerical ill-conditioning: the vectors a_i are reduced in length after successive subtraction of its projections.
- Householder's method is an alternative to Gram-Schmidt algorithms
- Key idea: introduce reflecting matrices of the form

$$H_i = I - 2w_i w_i^T \quad \text{with unit vectors } w_i \in \mathbb{R}^n, \quad \|w_i\|_2 = 1$$

- The matrices H_i are symmetric and orthogonal, since

$$H_i^2 = I - 4w_i w_i^T + 4w_i \underbrace{w_i^T w_i}_{=1} w_i^T = I$$

Householder's QR Algorithm: Derivation

- Let $a \neq 0$ be the first column of a matrix A .
- Our first goal is find a unit vector w such that

$$(I - 2ww^T)a \stackrel{!}{=} \lambda e_1 \quad \text{for a scalar } \lambda \in \mathbb{R}$$

- We claim that this is true for

$$w = \frac{a \pm \|a\|e_1}{\|a \pm \|a\|e_1\|}$$

- Proof: either use your geometric intuition or directly show that

$$\|a \pm \|a\|e_1\|_2^2 = 2(a \pm \|a\|e_1)^T a \implies (I - 2ww^T)a = \pm(-1)\|a\|e_1$$

Householder's QR Algorithm: Summary

- We can find $H_1 = I - 2w_1w_1^T$ such that

$$H_1A = \begin{pmatrix} \star & \star & \dots & \star \\ 0 & \star & \dots & \star \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \star & \dots & \star \end{pmatrix}$$

- We repeat the same procedure for all sub-blocks until

$$H_{n-1}H_{n-2}\dots H_1A = R$$

is upper triangular. Finally, $A = QR$ with $Q = H_1H_2\dots H_{n-1}$.